

Converting a Commodore 64 Keyboard into a USB HID with a Raspberry Pi

Introduction

The Commodore 64 (C64) was a popular 1980s home computer that integrated its motherboard and keyboard into one unit. Its keyboard, with 66 keys, includes a full complement of alphanumeric keys and special keys unique to the C64 (such as “Run/Stop” and the “Commodore” logo key) ¹. In this project, we repurpose the vintage C64 keyboard as a modern USB **Human Interface Device (HID)** using a Raspberry Pi as the interface controller. This report provides historical context on the C64’s keyboard design and explains how to interface it with a Raspberry Pi’s general-purpose I/O (GPIO) to emulate a USB keyboard. We discuss the hardware schematics and wiring needed (including a custom adapter board), the software strategy for scanning the key matrix and mapping it to USB key codes, and the challenges (debouncing, ghosting, timing) faced in recreating a responsive keyboard. We also compare this approach to other retro-keyboard conversion projects, highlighting the benefits of using a Raspberry Pi for flexible, embedded keyboard interfacing.

Background: The Commodore 64 Keyboard Design and Protocol

Keyboard Matrix Architecture: The C64 keyboard is a classic **8×8 matrix** of switch contacts, yielding 64 matrix positions (which correspond to 64 keys of the 66 physical keys) ¹. Each key in the matrix connects one of 8 “column” lines to one of 8 “row” lines. The keyboard’s internal PCB has printed traces grouping certain keys into common row or column circuits. When a key is pressed, it closes a connection between one specific row line and one specific column line. The matrix design greatly reduces the number of wiring lines needed: rather than 66 individual wires, the C64 keyboard uses 16 lines (8 rows + 8 columns) to detect all key presses. Figure 1 illustrates the concept of the matrix wiring: each switch connects a unique row/column pair, so that pressing a key effectively bridges one row line to one column line.

Figure 1: Commodore 64 keyboard connector pinout and matrix mapping. The C64 keyboard’s 20-pin connector brings out 8 column lines (handled by CIA#1 Port A bits PA0–PA7) and 8 row lines (CIA#1 Port B bits PB0–PB7). Two additional pins are for ground and the “Restore” key (plus one unused keyed slot and +5V not used by the keyboard).

Special Keys – Shift Lock and Restore: Of the 66 keys, 64 are wired in the matrix, while two keys are handled differently ¹. The **Shift Lock** key is a latching key that mechanically holds the left **Shift** down; electrically, it is simply wired in parallel with the left Shift switch. It has no unique electronic signal – engaging Shift Lock just forces the Shift circuit closed (behaving like a continuous Shift press). The **Restore** key is **not part of the matrix** ¹. On the C64, Restore is wired directly to the machine’s **NMI (Non-Maskable Interrupt)** line. Pressing Restore connects the NMI line to ground via a dedicated switch, causing a reset-like interrupt. For our conversion, the Restore key can be treated as a separate button (e.g. mapped to a modern key or special function), by reading its state on a GPIO input.

Original Scanning Mechanism: In the C64, the keyboard matrix is continuously scanned by the system's I/O chips (two 6526 CIA chips). One CIA (CIA #1) has its Port A lines (PA0–PA7) connected to the keyboard's column lines, and Port B (PB0–PB7) connected to the row lines. The C64's KERNAL (OS firmware) routine drives this matrix by **selecting one column at a time** and reading which row lines are pulled low. In practice, the CIA writes a byte to address \$DC00 (Port A) with a single 0 bit (low) selecting a specific column and 1s (high) on the other bits. It then reads back from \$DC01 (Port B) to see which row bits are 0, indicating a key pressed at the intersection of the driven column and those row(s). Each bit in the return indicates one of the 8 rows; a 0 means the key in the active column for that row is being pressed (the switch connects the currently-driven column line to that row input, pulling it low). The ROM scanning routine cycles through all 8 columns in this fashion thousands of times per second, so that any key press is quickly detected. (No interrupt is generated by keys – the process is purely polled in firmware). The only exception is the Restore key, which triggers an NMI interrupt; this key is handled separately from the matrix scan.

In summary, the C64 keyboard is a **passive switch network** – it contains no electronics or encoding logic of its own. All keys (except Restore) are read via the matrix scanning method. The keyboard connector carries ground, the matrix row/col lines, and the Restore line; notably, it does **not even need a power supply** for the keyboard itself. This simple but effective design will be replicated in our project by the Raspberry Pi, which will emulate the CIA's role of driving columns and sensing rows.

Objectives and Benefits of Repurposing the C64 Keyboard

Giving New Life to Vintage Hardware: Converting a Commodore 64 keyboard into a USB HID allows us to **reuse authentic retro hardware** as a modern input device. Many C64 units today are non-functional or spare parts; by salvaging the keyboard, enthusiasts can enjoy the **tactile feel and nostalgic layout** of a 1980s computer on present-day systems. This offers a unique typing experience – for example, the C64's springy keys and PETSCII graphic symbols on keycaps evoke the original era of computing. Repurposing the keyboard preserves a piece of computing history in a practical way, rather than letting it collect dust. It's especially appealing for retro-computing fans who may run C64 emulators: one can control the VICE emulator or other software using the *real C64 keys* instead of a generic keyboard. As one modder noted, turning retro machine keyboards into USB devices lets you **“use them with PC-based emulators, but still get the full *real computer* feel using the original hardware.”**

Technical and Educational Value: From an engineering perspective, this project is a valuable exercise in **embedded system design and interfacing**. It touches on multiple topics – GPIO usage, matrix scanning, keyboard protocols, and USB HID implementation. Hobbyists can learn about matrix keyboards (a concept still used in modern keyboards) and how to handle issues like key bounce or ghosting in software/hardware. Using a Raspberry Pi as the controller is convenient because it provides a Linux environment for development (making it easy to write and test code in C or Python), and it can directly interface with USB. The Raspberry Pi can effectively act as a “bridge” – reading the C64 key matrix and presenting itself to any PC as a standard USB keyboard. Unlike off-the-shelf adapters, a Pi-based solution is **fully customizable** – one can program custom key mappings, macros, or even use the Pi for additional tasks (for example, running a C64 emulator internally, so that the entire C64 case becomes a self-contained modern computer). The flexibility of software configuration is a major benefit.

Cost and Simplicity: Another goal was to achieve the conversion with minimal extra components. The C64 keyboard connects **directly to the Pi's GPIO header without need for intermediate microcontrollers or active converters** ². This keeps the hardware simple and low-cost. Commercial products like the *Keyrah*

interface board (which allows C64/Amiga keyboards to plug in as USB devices) or using an Arduino/Teensy microcontroller are alternative routes, but our approach avoids extra hardware by leveraging the Pi's I/O and processing. As noted in one project, a Raspberry Pi can be put inside a C64 case to handle the keyboard "without any extra hardware like Arduino or Keyrah". This not only reduces cost but also means fewer points of failure and a cleaner installation.

Hardware Design: Custom Interface Board and GPIO Wiring

Overview: To attach the C64 keyboard to the Raspberry Pi, we design a **custom interface board** that serves three main purposes: (1) adapt the C64's 20-pin keyboard connector to the Pi's 40-pin GPIO header, (2) provide stable power connections (5V and ground) to the Pi (and optionally to the C64's original power LED), and (3) facilitate a USB output connection from the Pi to the host computer. The interface board can be a simple PCB or prototyping board mounted inside the C64 case. It essentially routes signals from the keyboard connector to specific GPIO pins on the Pi and exposes a USB port for the Pi's USB gadget interface (if using a Pi Zero or Pi with OTG). Figure 2 shows a schematic-level view of the connections between the C64 keyboard and Raspberry Pi.

Figure 2: Simplified schematic of the interface between C64 keyboard and Raspberry Pi. The 8 column lines (from CIA Port A on the C64) and 8 row lines (CIA Port B) are connected to 16 GPIO pins on the Raspberry Pi (configured as outputs for columns and inputs for rows, respectively). The "Restore" key line is connected to a GPIO input with a pull-up. The Pi's ground is tied to the keyboard ground. If the original C64 power LED is used, it can be wired to a 3.3 V or 5 V source on the Pi through a suitable resistor ($\approx 330\text{--}390\ \Omega$). The custom board also breaks out a micro-USB port wired to the Pi's USB data lines for connecting to a PC, and a DC jack or USB power input to feed 5 V to the Pi (and optionally the LED).

Keyboard Connector Pinout: The Commodore 64 keyboard has a single-row 20-pin connector (0.1" pitch) that originally plugged into the C64 mainboard. Pin 1 of this connector is ground, and pin 2 is a blank (missing pin as an orientation key). Pin 3 carries the Restore key signal (normally open, connects to ground when Restore is pressed). Pin 4 provides +5 V from the C64 – however, the keyboard itself does not use +5 V, so this pin is not connected on the keyboard PCB. The remaining pins 5–20 are the 16 matrix lines. **Columns:** Pins 13, 19, 18, 17, 16, 15, 14, 20 correspond to column lines 0–7 (not in numeric order on the physical connector). **Rows:** Pins 12, 11, 10, 5, 8, 7, 6, 9 correspond to row lines 0–7 (again, ordered unconventionally). This mapping is based on the C64 schematic and confirmed by reverse-engineering efforts (the C64 Service Manual's diagram had some lines swapped, which later got corrected in community documentation). On our adapter board, we will wire each of these 16 lines to a designated GPIO on the Pi. It's wise to use color-coded wiring or silk-screen labels, because the ordering of pins on the connector is non-linear relative to the matrix indices.

For example, one feasible wiring assignment (following the connector pin numbering) to Raspberry Pi BCM GPIO pins might be:

- **Column 0** (connector pin 13) → GPIO 4
- **Column 1** (pin 19) → GPIO 17
- **Column 2** (pin 18) → GPIO 27
- **Column 3** (pin 17) → GPIO 22
- **Column 4** (pin 16) → GPIO 5

- **Column 5** (pin 15) → GPIO 6
- **Column 6** (pin 14) → GPIO 13
- **Column 7** (pin 20) → GPIO 19
- **Row 0** (pin 12) → GPIO 26
- **Row 1** (pin 11) → GPIO 16
- **Row 2** (pin 10) → GPIO 20
- **Row 3** (pin 5) → GPIO 21
- **Row 4** (pin 8) → GPIO 18
- **Row 5** (pin 7) → GPIO 23
- **Row 6** (pin 6) → GPIO 24
- **Row 7** (pin 9) → GPIO 25

This is just an example; the actual GPIO numbering is flexible, as long as we keep track in software. (One might choose contiguous GPIOs for each group to simplify programming, but it's not required.)

All **ground** connections must be common: we tie the keyboard connector pin 1 (keyboard ground) to the Raspberry Pi's GND pins (there are several on the 40-pin header). The **Restore** key line (pin 3 on the connector) goes to a GPIO input on the Pi. We configure this GPIO with an **internal pull-up** resistor so that it reads high normally, and will read low when the Restore key is pressed (shorting the line to ground). This mimics the original behavior where pressing Restore pulled the NMI line low. The +5 V pin from the keyboard connector (pin 4) is *not used* to power the keyboard, but we can optionally use it (or another 5 V source from the Pi) to drive the C64's **Power LED** if we retained it in the case. The original C64 power LED (in the power switch housing) can be wired to +5 V through a resistor ($\sim 390\ \Omega$) to the Pi's 5 V rail so it lights up when the Pi is powered.

GPIO Configuration and Levels: The Raspberry Pi's GPIO operates at 3.3 V logic. Fortunately, this is sufficient for the C64 keyboard matrix, which essentially consists of passive switches to ground. In our design, we will set the column lines as outputs driven either high or low, and set the row lines as inputs with pull-ups. The C64's original CIA chips used 5 V logic and internal pull-ups, but the keyboard does not actively drive any voltage – it simply connects lines to ground when keys close. Therefore, **no level shifting is required**; 3.3 V on the Pi's outputs and pull-ups works fine to detect key closures. Each keypress will connect a 3.3 V-driven line to 0 V, pulling an input low. We must ensure not to directly short two Pi outputs together – we will address this in the scanning method by never driving more than one column low at a time (others will either be high or set to high-impedance). The use of the Pi's **internal pull-up resistors** on input lines means we likely do not need external resistors for the matrix; this was similarly done on the original and in other projects to simplify wiring. For added safety, some builders insert $\sim 220\ \Omega$ series resistors on each matrix line to limit any short-circuit current during transitions or accidental misconfiguration (for example, if a software bug drove two crossing lines low and high causing a direct short through a pressed key). These resistors are optional but can provide protection without significantly affecting the logic levels due to the high-impedance nature of inputs and weak pull-ups.

Physical Board and Ports: The interface board can be made from a prototyping PCB. In one example build, a hobbyist **soldered a dual-row 40-pin female header** onto a perfboard for the Raspberry Pi's GPIO, and a **single-row 20-pin header** for the C64 keyboard connector, then wired the appropriate connections

between them. The board also needs to accommodate connections for power and the USB output. We can include a micro-USB jack (or a pigtail cable) connected to the Pi's USB OTG port (on a Pi Zero, the USB data port) so that the entire assembly can plug into a host PC as a USB keyboard. The Pi Zero's OTG supports device mode and can enumerate as an HID. If using a full-sized Raspberry Pi (which normally acts as USB host), an alternative is needed – e.g. using the Pi's Bluetooth or a serial-to-USB converter. For this project we assume a Pi Zero or Pi 4 in gadget mode for simplicity. The interface board might also feature a DC barrel jack or micro-USB for 5 V power input (wired to the Pi's 5V and GND pins). The **C64 case's original power switch** can even be repurposed to control the Pi's power (with some additional circuitry to safely shut down). In short, the custom board consolidates all needed connections: it is essentially an adapter and breakout for power and USB. Figure 3 (below) shows a photo example of an interface board in a C64 case that connects the keyboard and provides ports (from a similar project that used a Pi in a C64 without a Keyrah) – wiring is point-to-point between the keyboard connector and Pi GPIO, with an added panel for USB/power ports.

Figure 3: Example internal wiring (proto-board) for connecting a Commodore keyboard to a Raspberry Pi's GPIO (from a Plus/4 keyboard project). The 20-pin keyboard connector (right) is wired to the Pi's GPIO pins (via the ribbon cable to the Pi on the left). Such wiring can be done on perfboard or a custom PCB.

Electrical Considerations: The C64 keyboard lines normally idle high (pulled up) and are pulled low on key press. The Pi's internal pull-ups (typically ~50 kΩ) on input lines will keep them at 3.3 V minus any small leakage. When a column output is driven low, any pressed key in that column will tie the corresponding row input to ground. The internal pull-up is weak enough that it won't source excessive current into the low column – it just provides a reference high when open. All currents in this circuit are very small (essentially limited by the pull-up resistances and any brief transient capacitances). This means power consumption of the keyboard interface is negligible. The keyboard itself has no active power draw; even the Restore key is just a switch to ground. Thus, the only power considerations are the Pi itself and the optional LED. The Pi will typically be powered by a 5 V supply (which on Pi Zero can come through the same USB connector if acting as a device, or separately). We should ensure the ground from the host PC (through the USB cable) and the ground of the keyboard/Pi are common to avoid any ground potential differences. Additionally, proper strain relief and insulation should be applied to the wiring to avoid shorts inside the case.

Key Mapping Strategy for Modern Use

One challenge in converting a C64 keyboard is deciding how each C64 key should translate to a modern USB keyboard key code. The C64's layout and default characters differ from a standard PC keyboard, so we need a **keymap** that preserves usability. Our approach is to create a logical mapping that makes sense for both typical typing and for use in C64 emulation.

Alphanumeric Keys: For letters (A–Z) and digits (0–9), the mapping is straightforward – we map these keys to the corresponding USB HID key codes for the same characters. For example, pressing the C64 "A" key should register as the letter 'A' on the host, and so on. The C64 uses a positional layout akin to a typewriter, so the physical arrangement of letters and numbers matches the modern QWERTY layout (with some differences in symbol keys). We ensure that when shifted, the number keys yield the expected symbols (e.g. on a US keyboard, Shift+1 gives !). The C64 key legends for numbers were actually the graphics symbols (e.g. "£" on 1, "@" on 2, etc.), but for modern use we will interpret those keys in the conventional way (unless we are in a special emulator mode – more on that below). Essentially, we treat the C64 alphanumeric section as a normal keyboard for letters, numbers, and basic punctuation.

Modifiers: The C64 has two shift keys (left and right Shift) and a Control key (labeled “CTRL”), as well as the Commodore key (labelled with the **C=** logo, often used as an extra modifier in C64 software). We map the **Shift** keys to standard USB **Shift** modifiers (both left and right shift will be treated as Shift, perhaps distinguishing left vs right if desired for completeness). The **CTRL** key on the C64 can map to the modern **Control** key. The **Commodore (C=)** key has no direct modern equivalent – on the C64 it acted somewhat like an alternate graphics shift (and in BASIC it toggled upper/lowercase mode with Shift). For our mapping, we have a few choices: we could map Commodore key to, say, the modern **Alt** or **Windows/Super** key. Some projects choose to map the C= key to **Tab** or another harmless key in “ASCII mode”. For instance, one conversion project assigned Commodore key to Tab when using the keyboard as a normal input device. Another reasonable choice is to map it to the PC **Left GUI (Windows)** key, since that key often isn’t used by C64 programs but could serve as a special key in an emulator (e.g. many emulators use the Windows key or Alt for specific functions). In our design, we might map Commodore to the Left Alt or Left GUI key depending on user preference. The **Shift Lock** (Caps Lock) key, as discussed, behaves like a latched Shift on the C64. We have two options: (a) Map it to **Caps Lock** behavior on the host (i.e. toggle caps state when pressed), or (b) preserve original behavior by mapping it as a **Shift** key that is “held” when engaged. The latter is tricky in USB terms, because a locked physical switch doesn’t send a release. A compromise is to treat Shift Lock as CapsLock for text entry (so it toggles letter case without affecting number keys’ symbols), since that may be more usable for modern typing. But if the goal is an authentic C64 experience (e.g. in emulator), we might simply treat it as holding Shift down (which would, for example, produce symbols when number keys are pressed, just as on a real C64). For now, we can implement it as a Caps Lock toggle to align with typical modern behavior, acknowledging that it’s not exactly the same as the original.

Special Character Keys: The C64 had a variety of non-alphanumeric keys: **arrow keys** (printed as left-arrow and up-arrow on two keys), **function keys** (F1, F3, F5, F7 keys on the top row), **INST/DEL** (Insert/Delete) key, **CLR/HOME** (Clear/Home) key, **Return, Backspace** (actually labelled as “Delete” on C64, which acted like a backspace), **Escape** (run/stop can act like an escape in usage), etc. We assign these as follows:

- **Return:** Maps to **Enter** (HID Return) – straightforward.
- **INST/DEL:** The C64’s Insert/Delete key (when not shifted, it sends Delete character, when shifted it acted as Insert in software). We can map the unmodified key to PC **Backspace** (which deletes the character to the left, akin to C64’s delete). Optionally, Shift + INST/DEL could map to PC **Insert** (few people use Insert, but it’s one way to preserve the idea). In one documented mapping, they mapped C64 Inst/Del to Backspace, and Shift+Del to Insert.
- **CLR/HOME:** Unshifted, this key clears the screen (in C64) – we might map it to **Home** or **End** on PC. A reasonable mapping is: unmodified CLR/HOME → **Home** (go to beginning of line/page), and Shift + CLR/HOME (which is “CLR” on C64) → **End**. Alternatively, map the clear function to the **Clear** key (if available) or some combination. Some projects chose: C64 Home → PC Home, Shift+Home → PC End.
- **Run/Stop:** This key halted BASIC programs on the C64. On a modern system, it has no direct analogue. A common mapping is to map **Run/Stop** to **Escape**, since Escape is often used to stop or exit operations in modern software, and it suits the semantic of “stop.” Indeed, one Pico-based converter maps Run/Stop to ESC for ASCII mode use. We will do the same: C64 Run/Stop = ESC.
- **Arrow keys:** The C64 had two physical cursor keys: one printed with left-arrow and up-arrow, and the other with right-arrow and down-arrow (when combined with Shift, each key could produce the other member of the pair, since there were only two keys for four directions). For modern use, we prefer to present four distinct arrow keys. We can achieve this by detecting the use of Shift with the arrow keys and mapping to the corresponding directions. For example: without any modifier, have

the left-arrow key send **Left Arrow** and the right-arrow key send **Down Arrow**; if Shift is held, interpret left-arrow key as **Up Arrow** (since on C64 the up-arrow was printed on the same key as left-arrow) and the right-arrow key (with Shift) as **Right Arrow** ³. This way all four directions become accessible. Alternatively, we might ignore the C64's quirky shift requirement and simply map each physical key to two arrow outputs: left-arrow key always left, right-arrow key always right, and perhaps use Commodore or another modifier for up/down – but that complicates usage. The Shift method keeps it similar to original (C64 users are used to pressing Shift+arrow for the alternate direction). In an emulator context, often the arrow keys on a PC are mapped to the C64 directions in some way, so ensuring all four outputs exist is important. We will implement the shift-state check in software to emit the correct HID usage codes for the arrows.

- **Function Keys (F1, F3, F5, F7):** The C64 has four physical function key buttons, but labeled F1, F3, F5, F7. When pressed with Shift, these produced F2, F4, F6, F8 respectively in C64 software. For a modern HID, we have distinct key codes for F1 through F8 (and beyond). We will map the *unmodified* keys to F1, F3, F5, F7 as labeled, and decide how to handle the shifted versions. One approach is to detect the Shift modifier and when a function key is pressed with Shift, emit the adjacent function number. For instance, if the user presses Shift and the F1 key together, we can output **F2**. This allows the full range F1–F8 to be generated even though only four physical keys exist. Another approach seen in practice is to map the physical keys to the *lower* of the two numbers and have Shift modify it internally. For example, always treat the key labeled F1 as “F1” (and ignore the F2 possibility unless in emulator mode). But that means you couldn't get F2 at all from the keyboard unless you have special handling. It's better for completeness to implement the shifted variants: so our software will check: if a function key is pressed and the ShiftLock or Shift modifier is active, output the next function number. Concretely:

- Press F1 (no shift) → send HID **F1**.
- Press Shift+F1 → send HID **F2**.
- Press F3 → send **F3**; Shift+F3 → **F4**;
- Press F5 → **F5**; Shift+F5 → **F6**;
- Press F7 → **F7**; Shift+F7 → **F8**.

This way, all F1–F8 are accessible. (The C64 had no F9–F12, so those are not on the keyboard, although some modern emulators use F9–F12 for menus or fast reset. If desired, one could map some unused combo to F9–F12 – for example, one project used a triple-modifier chord for F9–F12, but that's beyond our scope here.)

Other Keys: The “=” key on C64 was on the top-right of main section (Shift+0 produced “=” on C64). We will map the key to = and + (with shift) as on a normal keyboard. The “£” key (on the 2 key in C64) will map to the “@” symbol on a modern US keyboard (Shift+2), since C64's default graphics on number keys don't directly translate. Essentially, we adopt a US ANSI keyboard layout for symbols when sending HID codes, so that the C64 keys output their nearest equivalent character. The mapping can be adjusted if using a different host keyboard layout (e.g., for UK layout, one might want to preserve the “£” symbol on Shift+3, etc.). For simplicity, assume US layout in the HID usage.

Finally, the **Restore** key can be mapped to a modern key or a special function. One idea is to map Restore to the “**Pause/Break**” key or to **F12** (since F12 is often unused in C64 context and could serve as a “reset” in an emulator). In one Pico project, they actually mapped Restore to a backslash “\” and Shift+Restore to “|” to provide those symbols, and used Shift+0 for F12 (the menu). There are multiple possibilities. We might choose to map Restore to **Break** (which is not commonly needed otherwise). If using the keyboard purely

for emulation, mapping Restore to an emulator reset or menu key (like F12) is very handy. For a more general-purpose use, Pause/Break or an unused scan code is fine. We will treat it as **Break** in our design, with the understanding that it could be remapped easily.

Summary of a Possible Keymap: To clarify the mapping, Table 1 below outlines each major C64 key and its intended HID output in this project:

- Letters **A-Z** → letters A-Z (case via Shift as normal).
- Numbers **0-9** → numbers 0-9; Shift modifies to !, @, #, \$, %, ^, &, *, (,) respectively (like a PC keyboard, not the C64's graphic symbols).
- **Comma (,)** →  (Shift for  as usual); **Period (. or >)** →  (Shift for .
- **Semicolon (;)** →  (Shift for .
- **Quote (')/Apostrophe:** The C64 had an "★" symbol on the quote key when shifted. We will map  (apostrophe) unshifted and  (double quote) shifted, to align with a normal keyboard's quote key behavior.
- **Slash (/)** →  (Shift for  if we want – though the C64 had  on Shift+/- already, so that matches).
- **Backarrow (←)** key (left of 1 on C64, printed as a left arrow symbol, often used as backspace in C64 BASIC): We map this to **Backslash (\)** or  key if we prefer. On a modern keyboard the key to left of '1' is backtick  (on US layout). The C64 had a "←" glyph there; in PETSCII this produced a left-arrow character. We might treat it simply as the **Esc** or **~** key. Some choose to map it to **Escape** as well, but we already used Run/Stop for Esc. Another approach: map C64 "←" key to the **Backquote** () key on a PC, so it produces a backtick (which somewhat resembles the C64 graphic). For now, we can set: **←** (backarrow key) → PC **~** (backtick/tilde). Indeed, one mapping has: **Up-arrow/Pi** key (the one next to right-shift on C64 that prints the Pi symbol) mapped to  and , and the left-arrow key mapped to delete. There are multiple variations; we will choose a reasonable one for demonstration.
- **Run/Stop** → **Escape** (as mentioned).
- **Control** → **Left Ctrl**.
- **Commodore (C=)** → **Left Alt** (or Win/Super – but we'll say Alt for now).
- **Shift (left & right)** → **Left Shift** and **Right Shift** (mapped distinctly for accuracy, though functionally similar).
- **Shift Lock** → **Caps Lock** (toggle).
- **Return** → **Enter**.
- **INST/DEL** → **Backspace** (and maybe Shift+Inst→Insert).
- **CLR/HOME** → **Home** (Shift+Clr→End).
- **Arrow keys:**
 - Physical "←/↑" key: without Shift -> **Arrow Left**; with Shift -> **Arrow Up**.
 - Physical "→/↓" key: without Shift -> **Arrow Down**; with Shift -> **Arrow Right**.
- **Function keys:** F1 key → **F1**, Shift+F1 → **F2**; F3 → **F3** (Shift+F3→F4); F5→F5 (Shift→F6); F7→F7 (Shift→F8).
- **Restore** → **Pause/Break** (for example).

This mapping ensures that for general computer use, the C64 keyboard will behave like a reasonable facsimile of a modern keyboard (letters and numbers work intuitively), while still allowing access to C64-specific keys if needed for emulators (e.g. Run/Stop as Esc, etc.). Enthusiasts can also create dual modes – an “emulator mode” where the mapping is one-to-one with C64 key matrix (generating PETSCII codes or

specific emulator triggers), and a “PC mode” where it behaves like a normal keyboard. In fact, one project provides two modes (MiSTer vs ASCII) and allows switching by a special key chord – but implementing such complexity is optional.

Software Implementation: Matrix Scanning and USB HID Emulation

With the hardware connected and a keymap defined, the core software task is to continuously **scan the keyboard matrix** via the Raspberry Pi’s GPIO and send the appropriate USB HID events for key presses and releases. We have two general approaches for the software: using user-space libraries on Linux (e.g. in Python or C) along with the Linux input subsystem, or implementing a bare-metal or kernel-space solution. We will describe a user-space approach using Python for clarity, which leverages the **RPI.GPIO** library for accessing GPIO pins and Python’s `evdev/uinput` module to emit keyboard events. This approach runs on Raspberry Pi OS (Linux) and treats the Pi itself as the “computer” that generates keyboard input (either for its own OS or via USB gadget). Alternatively, one could configure the Pi Zero as a USB HID gadget and write directly to the USB device endpoint; however, using the `uinput` interface is convenient for either case (if the Pi is in gadget mode, the uinput events can be routed to the USB gadget presenting as a keyboard).

GPIO Initialization: We designate 8 GPIO pins for columns (outputs) and 8 for rows (inputs). In code, we first set up the GPIO library and configure these pins. For example:

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM) # use Broadcom pin numbering
col_pins = [4,17,27,22,5,6,13,19] # example GPIO numbers for columns
row_pins = [26,16,20,21,18,23,24,25] # example GPIO numbers for rows
# Initialize rows as inputs with pull-ups, columns as inputs (high-Z) initially
for rp in row_pins:
    GPIO.setup(rp, GPIO.IN, pull_up_down=GPIO.PUD_UP)
for cp in col_pins:
    GPIO.setup(cp, GPIO.IN, pull_up_down=GPIO.PUD_OFF)
```

Here we set each row line as an input with an internal **pull-up** (`PUD_UP`), and each column line as an input with no pull (we will drive them as outputs one by one during scanning). Setting columns as inputs when idle effectively puts them in a Hi-Z state – similar to how the CIA chip would leave lines high (via pull-ups) except the one being driven low. This prevents any contention or accidental short when multiple keys are pressed (only one column will be actively driven at a time in our scan cycle). The Restore key pin would also be set as an input with pull-up (just like a row line).

Virtual HID Device Setup: Using Python’s `evdev` library, we can create a uinput device to emit keyboard events. For example:

```
from evdev import UInput, ecodes as e
ui = UInput(name="C64 Keyboard", vendor=0x1234, product=0x0001)
```

This sets up a virtual input device recognized by Linux. If the Pi is configured in USB gadget mode (using `dtoverlay=dwc2` and the `g_hid` driver or `gadgetfs`), these events can be directed to the host computer. Alternatively, if the Pi is the end-user device (say running an emulator locally), the events will just inject into the Pi's input system.

We also define our **keymap** in software – essentially an array or dictionary mapping each matrix [row, col] position to a specific `ecodes.KEY_*` value. For instance, using the mapping decided earlier, we fill a list of 64 entries (8 rows × 8 cols) where `index = row*8 + col`. For illustration, if row0 col0 is the "1" key, we set `index0 = KEY_1`; row0 col1 maybe is the "←" (left-arrow graphical, which we chose to map to backtick) so `index1 = KEY_GRAVE`; and so on for all keys. An excerpt (not the full map) might look like:

```
keymap = [
    e.KEY_1,      e.KEY_GRAVE,    e.KEY_LEFTCTRL, e.KEY_LEFTALT,  e.KEY_SPACE,
    e.KEY_LEFTMETA, e.KEY_Q,      e.KEY_2,      # Row0: 1, ←, Ctrl, C=, Space,
    Win(Commodore), Q, 2
    e.KEY_3,      e.KEY_W,      e.KEY_A,      e.KEY_LEFTSHIFT, e.KEY_Z,
    e.KEY_S,      e.KEY_E,      e.KEY_4,      # Row1: 3, W, A, L-Shift, Z, S, E, 4
    # ... (continue for all rows)
]
```

(In this snippet, we assumed matrix row0 includes keys "1, ←, Ctrl, Commodore, Space, Commodore (mapped to Meta/Super), Q, 2" just for example – actual C64 matrix ordering is different; one would populate it according to the known matrix layout ⁴. The key codes like `KEY_LEFTMETA` serve as the Commodore key mapping, etc.)

Matrix Scanning Loop: The program will run a loop to scan keys at a certain frequency (ideally fast enough for smooth typing, but not so fast as to waste CPU). A common choice is about **50–100 scans per second** (50–100 Hz). The C64 itself effectively scanned at the frame rate of the system (~60 Hz in NTSC), which is a good reference point. We will use ~60 Hz. In Python, a small `time.sleep()` can be used to pace the loop. At each iteration, we iterate through each column index 0–7:

1. Drive the current column line **low** (output 0 V) and set it as an output. All other column lines remain as inputs (high-impedance with pull-ups on the row side). For example, to activate column *i*:

```
GPIO.setup(col_pins[i], GPIO.OUT, initial=GPIO.LOW)
```

Now that column's line is actively pulled down. The row inputs will read **LOW (0)** for any row where the corresponding key in this column is being pressed (because that row line is now electrically connected to the low column through the closed key switch). Unpressed keys leave the row line pulled up (HIGH).

2. Read all row input states. We loop through `j = 0..7` for each row pin:

```
state = GPIO.input(row_pins[j])
```

If `state` is `GPIO.LOW`, it means the key at [row j, col i] is pressed. If `state` is `GPIO.HIGH`, that key is not pressed (or that row is not pulled down in this column).

3. For each key position, we maintain a record of its last known state (pressed or not pressed) – e.g. using a Python set or a boolean array. We detect **state changes**: if a key that was previously not pressed is now LOW, we register a **key press event**; if it was pressed and now went HIGH, we register a **key release event**. We then update the stored state accordingly.
4. After scanning the rows for that column, we **release the column line** by setting it back to input (hi-Z with no pull or with pull-up – but since row lines have pull-ups, we can just set it to input mode). For example:

```
GPIO.setup(col_pins[i], GPIO.IN, pull_up_down=GPIO.PUD_OFF)
```

This ensures that when we move to the next column, we don't have two outputs driving different levels.

5. Move to the next column and repeat.

At the end of each full matrix scan (all columns 0–7), we can insert a short delay to control the loop timing (e.g. sleep for ~1/60 s). It's important to also send the accumulated key events out via HID. In the `evdev` approach, we would call `ui.syn()` to synchronize events after we've enqueued all key presses/releases for that cycle.

Below is a simplified pseudocode (in Python style) for the scanning loop, integrating the above steps and using the `uinput` device for output:

```
pressed_set = set() # to keep track of currently pressed key indices
SCAN_INTERVAL = 1/60 # 60 Hz
while True:
    syn_needed = False
    for i, col_pin in enumerate(col_pins):
        # Drive column i low
        GPIO.setup(col_pin, GPIO.OUT, initial=GPIO.LOW)
        # Check all rows for pressed keys in this column
        for j, row_pin in enumerate(row_pins):
            idx = i*8 + j # unique index for key at (col=i, row=j)
            if GPIO.input(row_pin) == GPIO.LOW:
                # Key is physically down
                if idx not in pressed_set:
                    pressed_set.add(idx)
                    ui.write(e.EV_KEY, keymap[idx], 1) # key down event
                    syn_needed = True
            else:
                # Key is up (not pressed)
```

```

        if idx in pressed_set:
            pressed_set.remove(idx)
            ui.write(e.EV_KEY, keymap[idx], 0)    # key up event
            syn_needed = True
        # Release column i (back to input high-Z)
        GPIO.setup(col_pin, GPIO.IN, pull_up_down=GPIO.PUD_OFF)
    # Sync events if any key state changed in this cycle
    if syn_needed:
        ui.syn()
    time.sleep(SCAN_INTERVAL)

```

This logic closely mirrors what the original C64 did in hardware/firmware, but now in our Pi software. In fact, a very similar structure is seen in existing projects. For instance, the loop runs at 60 Hz and for each row (they chose rows to drive and read columns, which is equivalent just transposed) it sets that line LOW, reads all others, and uses a set to track pressed keys. Our code uses `ui.write(EV_KEY, code, value)` to output key events, where `value` is 1 for key press and 0 for release. The `uinput` system will handle combining these into USB reports if the Pi is in gadget mode, or injecting into the Linux input subsystem if used locally.

USB HID Output: The Raspberry Pi, when configured properly, will appear as a USB keyboard to any host it's plugged into. Each scan cycle, when we call `ui.syn()`, the `uinput` device emits the buffered key events. This ultimately results in USB HID reports being sent, containing the keycodes of all currently pressed keys (and the status of modifiers). The HID report format for keyboards typically includes up to 6 key codes for concurrent presses plus a bitmap for modifiers. Our approach inherently handles multiple keys: the `pressed_set` can accumulate several keys if they are held down simultaneously, and we generate individual press events for each. If more than 6 keys are held (rare in typing), some USB limitations might drop them, but typical use won't exceed this.

Debounce Handling: Mechanical key switches like those in the C64 can **bounce**, i.e. produce rapid on/off fluctuations for a few milliseconds when pressed or released. In our simple loop, a bouncing key might register as multiple press/release changes in consecutive scans. For example, a single key press could generate a sequence like: LOW on first read (key pressed) → HIGH on second read (bounce open) → LOW again on third (settled pressed). This could lead our logic to think the key was pressed, then released, then pressed again in quick succession. To mitigate this, we can incorporate a debounce filter. A straightforward solution is to only declare a state change if the new state persists for a certain number of consecutive scans or a minimum time. Since we scan at 60 Hz (~16.7 ms per scan), even 2 consecutive reads can cover ~33 ms, which is typically enough to ride out a bounce. A simple debounce could be: when a transition is detected, delay action for one scan to confirm it. However, in practice many projects omit explicit debouncing and rely on the key press duration being longer than one scan cycle. The Python code above does *not* explicitly debounce – it would act on a bounce. We could refine it by storing a timestamp for each last change and ignoring reversals that happen within, say, 20 ms. Another option is scanning at a slower rate (like 100 Hz) and that naturally smooths out very brief flickers. For an academic thoroughness, one could implement a small state machine per key: e.g., require 2 consecutive LOW reads before accepting a key-down, and similarly 2 consecutive HIGH reads before key-up. This would eliminate most bounce artifacts while still scanning fast. In performance-critical C code, one might use a matrix of counters for this purpose.

Example in C (overview): If using C or C++ for better performance, the logic is analogous. One would `mmap` or use `sysfs` / `libgpiod` to access GPIO or use the WiringPi library, etc. Then set DDR (data direction) registers accordingly for outputs/inputs. The scanning could even be done in a Linux kernel module or bare-metal on a microcontroller (like in the PIC-based project or a Pi Pico RP2040 using the TinyUSB stack). Since our focus is on concept, we stick to the Python approach which is easier to read. It's worth noting that a Python implementation at 60 Hz uses only a few percent CPU on the Pi, whereas a C implementation would be <1% CPU. Both are easily manageable on a modern Pi.

Verifying the Key Mapping: We should test that each key press sends the intended character. For instance, pressing the C64 "A" key should result in an `'a'` on the host. If it yields a different letter, likely the matrix index mapping is off. Using a test program or observing evdev events can help verify the mapping table. It's useful to print debug info of which key index is detected or even light an LED when certain keys are pressed (for example, connect the CapsLock LED output event to the C64's LED to replicate that functionality).

In summary, the software turns the Raspberry Pi into a keyboard controller that continually scans the matrix and issues HID reports. This is fundamentally similar to what an Arduino running the HID library or a dedicated keyboard controller firmware (like QMK) would do, but we leverage the Linux environment of Pi for ease of development.

Challenges and Considerations

Designing this C64-to-USB keyboard interface comes with a few **technical challenges**, which we address through both hardware and software strategies:

- **Key Debouncing:** As mentioned, physical switch bounce can cause false multiple triggers. If untreated, a quick tap on a key might register as two or more key presses. To ensure a **clean typing experience**, one must debounce. Our software scanning at 60 Hz inherently filters out very high-frequency bounce (anything shorter than ~16 ms might be missed between scans), but bounce can last on the order of 5–10 ms. By slightly slowing the scan or adding a verification delay, we can avoid most issues. Another approach is to implement a **delay after detection** – e.g., once a key transition is seen, ignore further changes on that key for the next, say, 20 ms until the contact settles. In a Python loop, this could be done with timestamps; in C, with counters or timers. Fortunately, typing generally involves keys being held for tens of milliseconds at least, so bounce is not as critical as it would be in, say, a one-shot button context. Nonetheless, for professional polish, we include debouncing logic.
- **Ghosting and Masking:** In keyboard matrices **without diodes**, certain combinations of simultaneous key presses can cause phantom signals (ghost keys) or masking of some presses. **Ghosting** occurs when, for example, two keys in the same row and two keys in the same column are pressed at once – the fourth corner of that rectangle can erroneously appear “pressed” because current can flow through the multiple closed switches. The C64 keyboard did *not* have diodes on each switch, so it is vulnerable to ghosting if three specific keys form a rectangle in the matrix. In practice, typists rarely press such combinations unintentionally, but it can happen (especially if holding a modifier and pressing two other keys that align in matrix). To mitigate ghosting, one solution is to add **diodes in series** with each key switch (which is infeasible inside an existing keyboard without major modification), or to implement **2-key rollover** limit in software (i.e., if a ghost condition is detected, ignore or do not register the phantom). In our Pi scanning, ghosting

would manifest as a row read low in a column that wasn't actually pressed. There is a technique to detect ghosting: if multiple keys are down and they share both a row and column with a third, then ghost may be present. But implementing full ghost avoidance in software is non-trivial without hardware diodes. A more practical approach is to simply document that certain 3-key combinations may not be recognized distinctly. However, because the C64 matrix was designed around typical typing patterns and included the restriction that two specific keys (Shift Lock and Restore) aren't in matrix, the likelihood of ghosting interfering with normal use is low. If a stricter requirement for rollover is desired (e.g. for gaming, pressing many keys at once), one might consider hardware modifications (diodes) or use a different scanning algorithm. One interesting mitigation used in other projects is to drive outputs not as actively high, but use pull-ups and only pull one line low at a time (which we do), *and* treat multiple key presses on different columns carefully. In our design, when one column is driven low, all other columns are high-impedance (pulled up only via inputs). This means if two keys in different columns and one key in their same row are pressed, the shared row will connect to two column lines (one actively low, one pulled high via resistor). The result is the pulled-up column might get partially pulled low through the switches – it could register as low since the pull-up is weak. This is ghosting: a key in the non-selected column appears pressed. By using high-impedance for non-driven columns, the current is limited by the pull-up resistors but still enough to register. **Hardware fix:** If we had diodes on each column line at each switch, pressing multiple keys wouldn't back-feed into other columns. Lacking diodes, one partial software remedy is to detect the condition: if we find two keys down in different columns and different rows, and a third reading suggests a phantom, we could ignore the phantom. This requires scanning in such a way to identify ghosts. Many NKRO (n-key rollover) keyboard controllers solve it with diodes; we will assume typical use and/or minimal ghosting issues for simplicity. In testing, we might find the specific combinations that ghost (for instance, on C64 pressing Q and E and Shift together might ghost the W, etc., just hypothetically). The ghosting issue is inherent to the matrix; the original C64 accepted that limitation (for example, pressing certain three-key combos on a real C64 produces a phantom character). For an improved design, adding diodes on the adapter board is possible: one could route each column line through a diode array (with cathodes at the keyboard side for example) to prevent current from flowing from one column into another via a shared row. Some modern implementations did exactly this when using microcontrollers. If ghosting becomes problematic, one can incorporate 1N4148 diodes in series with each column line on the custom PCB. In our build, we note this as an optional enhancement for full n-key rollover support.

- **Latency and Timing:** Using a Raspberry Pi running Linux means we do not have a real-time guarantee for our scanning loop. High system load or scheduling could in theory cause the scanning to jitter in timing. In practice, a 60 Hz loop in Python is lightweight, but if the Pi is also, say, running an emulator or other tasks, we should consider performance. A **C implementation or running on bare-metal** would give more deterministic timing and lower latency. Linux's event handling and the USB stack add some minimal latency (~a few milliseconds). Overall, a well-tuned Pi can easily achieve <20 ms key-to-USB latency, which is on par with or better than many USB keyboards. Our scanning interval (16.7 ms) plus processing is the dominant factor. If we needed to improve this, we could increase scan rate (e.g. 200 Hz) or optimize code. The C version of a similar driver uses <1% CPU at 60 Hz, so we could even bump to 500 Hz with little issue if needed. However, very high scan rates are unnecessary for human typing (even a 30 Hz scan can capture a 50 ms key tap). We chose ~60 Hz to balance responsiveness with CPU usage. The Raspberry Pi's speed means we have *excess* capability here; even interpreted Python handled it fine with single-digit CPU%. In more demanding scenarios (like scanning not just keyboard but also joysticks, etc., as some projects did), one might integrate

the scanning into a C program or the kernel. Another timing consideration is the USB polling rate: by default, USB HID keyboards poll at 125 Hz (every 8 ms). So even if we scan faster, the host will see updates at 8 ms intervals. This is usually fine; if needed, one can configure the HID descriptor for faster polling, but 8 ms (or 1/125 s) is already plenty smooth for typing.

- **Power and Safety:** The Pi powering the keyboard is straightforward, but we must be cautious about how we supply 5 V when using gadget mode. If the Pi is plugged into a host via USB, typically the host will provide 5 V power over USB. The Pi Zero can be configured to accept power from the host's 5 V while simultaneously acting as a device. Our custom board's power input should accommodate this – essentially if powered via the USB cable from the PC, that feeds the Pi's 5 V rail which can also drive the LED, etc. If instead an independent supply powers the Pi (like a wall adapter), and the Pi is also connected to a PC's USB, we must ensure not to backfeed power into the PC. In that scenario, using a **data-only USB cable** or a power isolation on the USB lines might be needed. It is an electrical detail beyond the keyboard itself but important for deployment. Also, since the keyboard connector pins are directly wired to CPU GPIOs, **ESD protection** is a consideration. The original design left those lines somewhat exposed (e.g. user port and joystick ports also tied into CIA, risking static shocks). In a modern remake, we could add small TVS diodes or series resistors. At minimum, the user should avoid touching the keyboard connector pins or inserting/removing the keyboard cable while powered, to prevent electrostatic discharge from zapping the Pi's SoC. The risk is low if everything is enclosed in the case.
- **Mechanical Fitting:** Although not an electrical issue, it's worth noting the **mechanical aspects:** The Raspberry Pi (especially a Pi Zero or a Pi 3A/3B) can fit inside the C64 case with room to spare. The custom board should be shaped to mount near where the original keyboard cable reached the mainboard. One can use the original mounting points or standoffs to secure the Pi and adapter. In one example, a hobbyist drilled holes in a protoboard to align with the Pi's mounting holes and mounted the board above the Pi, with the keyboard connector attached. Care must be taken to not strain the 35+ year old keyboard connector or cable. Many builders actually desolder the flimsy connector from the old motherboard and use it on the interface board for a clean plug-in of the keyboard. If the original connector is unavailable, a 20-pin single-row header (2.54 mm pitch) can mate with the keyboard cable, but it may not lock – some have 3D-printed a small holder to secure it. Our design assumes we have a proper connector to ensure a good electrical connection and easy assembly.

Comparison with Other Retro-Keyboard Conversion Projects

Converting vintage keyboards to USB is a popular endeavor, and various methods exist:

- **Microcontroller-based Converters:** A common approach is to use a dedicated microcontroller (such as an Arduino Pro Micro (ATmega32u4), Teensy, or Raspberry Pi Pico RP2040) to interface with the keyboard matrix and act as a USB HID device. For the C64 keyboard, projects like Simon Inns' "C64 USB keyboard front-end" used a PIC18F4550 microcontroller, and others have used Arduino Pro Micro (ATmega32u4) which natively supports USB HID. The microcontroller approach typically involves writing firmware (using libraries like LUFA or Arduino's Keyboard library) to do the scanning and USB reporting. The advantages are low latency, simplicity (the microcontroller can be placed on a small board that plugs right into the keyboard connector), and no OS overhead. For instance, the Bytes<N>Bits project demonstrates using an Arduino Pro Micro with diodes added for each row line

to avoid ghosting, and the Arduino's internal pull-ups and USB library to enumerate as a keyboard. Our Raspberry Pi method achieves the same end result, but the Pi is arguably overpowered for just a keyboard interface – however, the Pi offers ease of programming (no need to flash firmware via ISP, etc.) and the possibility to do more than just keyboard interfacing (like hosting an emulator, driving a display, etc. simultaneously).

- **Commercial Adapters (Keyrah):** The *Keyrah* is a well-known product that provides an interface for Commodore 64 (and VIC-20, etc.) keyboards to USB. It's essentially a PCB with a microcontroller that the C64 keyboard plugs into, and it outputs USB. Keyrah also supports connecting the C64's joystick ports. The Keyrah requires no programming by the user – it's plug and play. Our approach with the Pi is DIY and offers more customization, but requires significantly more work and additional power considerations. One reason to prefer the Pi over Keyrah could be if you plan to embed a full Raspberry Pi system in the C64 (to also emulate the computer), in which case using the Pi's GPIO directly saves space and cost (no separate adapter). As noted earlier, using the Pi means *"no extra hardware like Arduino or Keyrah"* is needed – the GPIO pins take their place.
- **Other Retro Keyboards:** Similar conversions have been done for other classic computers – e.g. the IBM Model M keyboards (which use a matrix + controller, often replaced with a modern controller running custom firmware), Apple II/IIE keyboards to USB, or even typewriter conversions. Many of those use either a Teensy running **QMK firmware** (an open-source keyboard firmware supporting custom matrices) or Raspberry Pi Pico (with CircuitPython or TinyUSB). For example, the **"Keymmodore"** project uses a Raspberry Pi Pico (RP2040) as a drop-in replacement controller for a C64, running custom firmware to read the matrix and present USB – essentially a microcontroller solution with Pi Pico branding. The Pico, being \$4 and having PIO/state machines, is an elegant solution too – but it requires writing/compiling C or using CircuitPython (Adafruit even published a guide for a Commodore 16 keyboard using CircuitPython's USB HID library). Compared to a full Raspberry Pi, microcontrollers boot instantly and have deterministic timing; a Pi Zero running Linux takes a few seconds to boot and depends on the OS to be stable. In an "embedded keyboard" context, a microcontroller might be considered more *"professional"* for just keyboard duty. However, if the goal is to integrate *computing* into the C64 (to make it a standalone retro-computer), then a Pi is advantageous as it can run the emulator and handle the keyboard input internally.
- **Feature Comparison:** Using a Raspberry Pi for the keyboard gives the ability to easily **log keystrokes, add macro capabilities, or network the input** (imagine sending key events over WiFi/Bluetooth as well). It's essentially a small Linux computer doing the job. This might be overkill for some, but it offers flexibility. Dedicated microcontroller solutions are more constrained but very efficient at just being a keyboard. Another project by *WaitingForFriday.com* used a PIC and achieved a composite USB device (keyboard + two joysticks) for a C64 front-end – the Pi could also do that (one could use Pi GPIO for joysticks too and combine into one USB device descriptor). On a Pi running Linux, creating a composite HID (keyboard + gamepad) might require configuring the USB gadget with multiple interfaces, which is doable through configs or libcomposite. So, the Pi is quite capable of matching those features.
- **Key Mapping in Other Projects:** Many converters have had to tackle the mapping of special keys. Our approach is in line with common solutions: Run/Stop to Esc, etc. For example, the Pico-based `cbm2usb` project provides a **VICE-specific keymap** file for symbolic mapping in the emulator, and an ASCII mode for general use. They solved certain tricky keys (like C64's British Pound key and the

line-feed key on VIC-20) by mapping to rarely used PC keys and providing an alternate mapping. They also mapped the combination of both Shift keys + Ctrl as a special command (like Ctrl+Alt+Del equivalent). In comparison, our project's mappings aim for simplicity and likely won't implement such complex chords.

In summary, using the Raspberry Pi is a **modern, software-centric approach** to this conversion. It may not be as minimalist as using a tiny microcontroller, but it leverages accessible software tools and the Pi's ecosystem. This approach demonstrates how embedded Linux can handle real-time-ish I/O tasks given the speed of the Pi. It also allows the project to be expanded beyond just a keyboard – for instance, one could integrate an OLED display or use the Pi to act as a mass storage device for the host at the same time, etc., though those are outside our current scope.

Conclusion and Further Insights

Transforming the Commodore 64's keyboard into a USB HID device has allowed us to blend **retro hardware charm with modern functionality**. The historical design of the C64 keyboard – an 8×8 matrix scanned by the computer – can be faithfully replicated using the Raspberry Pi's GPIO lines and a bit of code. We detailed how each key is wired in the matrix and how the Raspberry Pi can systematically drive and read those lines to detect key presses, just as the original C64 did via its CIA chips. The custom interface board we described serves as a bridge between two eras: it connects 1982-era hardware to a 2025-era microcomputer and USB interface, while providing necessary amenities like power input and a USB connection.

Through this project, we revisited fundamental concepts of **embedded systems and digital design**. We addressed **key matrix scanning**, which is a technique still relevant in today's keyboards (modern mechanical keyboards use similar matrix scanning, often with diode isolation for n-key rollover). We dealt with **contact debouncing**, a classic problem whenever reading mechanical switches. We also confronted **ghosting** issues inherent in matrix keyboards without diodes – a problem that engineers have solved in various ways over the decades. Our solution noted the trade-offs and provided options (from adding diodes to accepting limited rollover) in the context of the C64 keyboard.

From an electrical standpoint, we showed that the C64 keyboard's passive switch network is easy to interface – it operates with simple pull-up resistors and ground signals, which the Raspberry Pi can handle directly at 3.3 V logic. The importance of internal pull-ups was highlighted, just as the original C64 relied on the CIA's internal pull-ups. We also ensured that the **Restore key** (NMI) wasn't forgotten, giving it a role in the USB mapping, and considered the **Shift Lock's** unique behavior in a modern context.

By implementing the keyboard scanning in Python (for clarity), we proved that even a high-level language on Linux can manage real-time keyboard duties, thanks to the speed of modern processors and relatively low required scan rates. For a truly professional product, one might port this to a C program or even a bare-metal microcontroller firmware, but the end result would be the same key events flowing over USB. It's a testament to how far computing power has come – a task that consumed a noticeable fraction of a 1 MHz 6502's time (the C64 polled keyboard in its IRQ loop) can now be done as a background task on a 1 GHz Pi with negligible load.

One *insight* gained is the versatility of the Raspberry Pi's Linux environment for hacking devices: by using `/dev/uinput` (via the evdev library) we seamlessly turned GPIO inputs into a virtual HID without writing

kernel drivers from scratch. This showcases a **rapid prototyping advantage** – we can modify key mappings or add features by simply editing a Python script and restarting it, rather than re-flashing firmware. The development feedback loop is faster, which is valuable in fine-tuning the user experience (for example, adjusting debounce times or remapping a key if we find the original choice awkward).

Another insight is the importance of **preserving the user experience** of the old hardware while integrating with new systems. For instance, we took care to map keys in a logical way so that a C64 touch-typist would be comfortable, yet a modern application would recognize the inputs. This often required compromise (Shift Lock vs Caps Lock behavior, etc.), but our structured approach means these decisions are documented and could be adjusted if a user desires a different behavior. For example, an alternate mapping mode could be coded and toggled by a special key chord – a feature that could be added thanks to the flexibility of the Pi.

In comparing to other projects (whether using Arduino or commercial boards), we learned that while many roads lead to Rome, each has its nuances. A microcontroller solution might yield slightly lower latency or simpler hardware, while a Pi solution offers ease of programming and extensibility. There is no single “correct” method – it depends on the project scope. If one were mass-producing a C64-to-USB kit, a small microcontroller would be cost-effective. But for an individual hobby build, repurposing a Pi you have on hand can be just as effective and more educational. Additionally, if the aim was not just a keyboard but a full C64 retrofit (with Pi running an emulator), then using the Pi’s GPIO is clearly the preferred path, as evidenced by projects like BMC64 (Bare Metal C64) which directly support reading the keyboard via GPIO for exactly this reason.

In conclusion, our Raspberry Pi-based C64 keyboard converter achieves the goal of a **professional-grade USB keyboard interface**. We interfaced with the old hardware at the electrical level, wrote software to interpret it, and integrated it with the USB HID protocol to communicate with any modern computer. The end result allows one to *type on a Commodore 64 keyboard and have the input recognized by a contemporary PC or Raspberry Pi*, which is a delight for retro-computing enthusiasts. By carefully addressing technical challenges like bouncing and ghosting, and by employing a thoughtful keymap, we ensured that the typing experience is reliable and intuitive. This project not only extends the life of classic hardware but also stands as a practical exercise in combining old and new technology – a small case study in the evolution of interface design from the 1980s to today’s plug-and-play USB world.

References: This report was informed by numerous sources, including the C64’s schematics and documentation of its keyboard matrix, community-driven reverse engineering efforts that provided corrected wiring maps, and modern projects that inspired the methods used (such as the use of Raspberry Pi GPIO with user-space drivers, and the incorporation of diodes to prevent ghosting in other keyboard conversions). Each aspect of the design was cross-verified with these references to ensure accuracy and robustness. The successful implementation demonstrates the soundness of using primary historical data combined with contemporary engineering practices to achieve the desired outcome.

1 3 4 Keyboard - C64-Wiki

<https://www.c64-wiki.com/wiki/Keyboard>

2 GitHub - int-0/c64keyboard: Tools to connect C64 keyboard to a Raspberry GPIO port

<https://github.com/int-0/c64keyboard>